Homework 09 Solutions

Prerith Arunkumar (prerith2)

Choosing an SVM kernel for the South African heart disease data by five-fold cross-validation, and a short hand-written SVM analysis skill applied to the same data.

Use of AI tools. I used Claude (Claude Code) to draft the `R` code and exposition for Question 1 and to run it. Every number below comes from the code output shown in this report, and I checked the numbers quoted in the text against it. For Question 2, I wrote `SKILL.md` by hand without AI assistance; as the assignment requires, an AI agent applied the skill to produce the one-page report, which I then checked.

Question 1: Choosing an SVM kernel

```
suppressWarnings(library(e1071))
heart <- read.csv("data/saheart.csv")
heart$famhist <- ifelse(heart$famhist == "Present", 2, 1)   # Absent = 1, Present = 2
X <- as.matrix(heart[, setdiff(names(heart), "chd")])
y <- factor(heart$chd)
n <- nrow(X); p <- ncol(X)
c(n = n, p = p, chd_rate = mean(heart$chd))
```

```
          n           p    chd_rate
462.0000000   9.0000000   0.3463203
```

```
set.seed(432)                                   # stratified 5 folds, shared by all
 ↪  candidates
fold <- integer(n)
for (cls in levels(y)) {
  idx <- which(y == cls)
  fold[idx] <- sample(rep(1:5, length.out = length(idx)))
}
table(fold, y)
```

```
    y
fold  0  1
   1 61 32
   2 61 32
   3 60 32
   4 60 32
   5 60 32
```

Candidates (19). Linear: $C \in \{0.01, 0.1, 1, 10\}$. Polynomial of degree 2, 3, 4: $C \in \{0.1, 1, 10\}$ each, with the kernel $(\gamma x^\top x' + r)^d$ held at $\gamma = 1/p = 1/9$ and $r = 1$. Radial $\exp(-\gamma\|x - x'\|^2)$: $C \in \{0.1, 1, 10\}$ and $\gamma \in \{0.01, 0.1\}$. All fits use `e1071::svm` with `scale = FALSE`, after standardizing each fold's fitting observations and applying their means and standard deviations to the held-out fold.

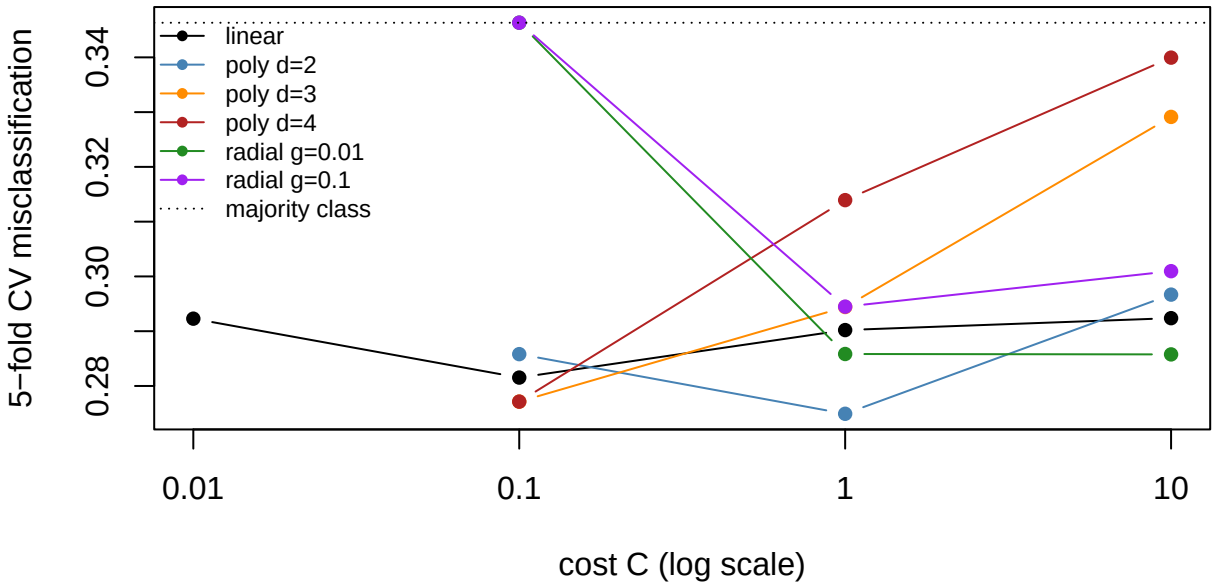
```r
cands <- rbind(
  data.frame(kernel = "linear",     degree = NA, gamma = NA,   cost = c(0.01, 0.1, 1, 10)),
  expand.grid(kernel = "polynomial", degree = 2:4, gamma = 1 / p, cost = c(0.1, 1, 10),
              stringsAsFactors = FALSE),
  expand.grid(kernel = "radial",     degree = NA, gamma = c(0.01, 0.1), cost = c(0.1, 1, 10),
              stringsAsFactors = FALSE))
stopifnot(nrow(cands) <= 20)

fit_one <- function(Xtr, ytr, Xte, cand) {
  args <- list(x = Xtr, y = ytr, kernel = cand$kernel, cost = cand$cost, scale = FALSE)
  if (cand$kernel == "polynomial") args <- c(args, degree = cand$degree, gamma = cand$gamma,
    ↪  coef0 = 1)
  if (cand$kernel == "radial")     args <- c(args, gamma = cand$gamma)
  predict(do.call(svm, args), Xte)
}

err <- matrix(NA, nrow(cands), 5)
for (k in 1:5) {
  tr <- fold != k
  mu <- colMeans(X[tr, ]); s <- apply(X[tr, ], 2, sd)
  Xtr <- scale(X[tr, ], mu, s); Xte <- scale(X[!tr, ], mu, s)
  for (j in seq_len(nrow(cands)))
    err[j, k] <- mean(fit_one(Xtr, y[tr], Xte, cands[j, ]) != y[!tr])
}
cands$cv_error <- rowMeans(err)
cands$se <- apply(err, 1, sd) / sqrt(5)
cands$label <- with(cands, ifelse(kernel == "linear", "linear",
                    ifelse(kernel == "polynomial", paste0("poly d=", degree),
                           paste0("radial g=", gamma))))
knitr::kable(cands[, c("kernel", "degree", "gamma", "cost", "cv_error", "se")],
             digits = 3, row.names = FALSE)
```

kernel	degree	gamma	cost	cv_error	se
linear	NA	NA	0.01	0.292	0.011
linear	NA	NA	0.10	0.282	0.016
linear	NA	NA	1.00	0.290	0.017
linear	NA	NA	10.00	0.292	0.018
polynomial	2	0.111	0.10	0.286	0.012
polynomial	3	0.111	0.10	0.277	0.009
polynomial	4	0.111	0.10	0.277	0.011
polynomial	2	0.111	1.00	0.275	0.009
polynomial	3	0.111	1.00	0.294	0.008
polynomial	4	0.111	1.00	0.314	0.017
polynomial	2	0.111	10.00	0.297	0.018
polynomial	3	0.111	10.00	0.329	0.013
polynomial	4	0.111	10.00	0.340	0.017
radial	NA	0.010	0.10	0.346	0.001
radial	NA	0.100	0.10	0.346	0.001
radial	NA	0.010	1.00	0.286	0.015
radial	NA	0.100	1.00	0.294	0.011
radial	NA	0.010	10.00	0.286	0.008
radial	NA	0.100	10.00	0.301	0.017

```
groups <- unique(cands$label)
cols <- c("black", "steelblue", "darkorange", "firebrick", "forestgreen", "purple")
par(mar = c(4, 4, 1, 1))
plot(NA, xlim = range(log10(cands$cost)), ylim = range(cands$cv_error), xaxt = "n",
     xlab = "cost C (log scale)", ylab = "5-fold CV misclassification")
axis(1, at = log10(unique(cands$cost)), labels = unique(cands$cost))
for (g in seq_along(groups)) {
  s <- cands[cands$label == groups[g], ]
  lines(log10(s$cost), s$cv_error, type = "b", pch = 16, col = cols[g])
}
abline(h = mean(heart$chd), lty = 3)              # always predict "no CHD"
legend("topleft", legend = c(groups, "majority class"), col = c(cols, "black"),
       lty = c(rep(1, length(groups)), 3), pch = c(rep(16, length(groups)), NA), cex = 0.75,
       ↪  bty = "n")
```




```
best <- cands[which.min(cands$cv_error), ]
one_se <- cands[cands$cv_error <= best$cv_error + best$se,   # within one SE of the best
                c("kernel", "degree", "gamma", "cost", "cv_error")]
best[, c("kernel", "degree", "gamma", "cost", "cv_error", "se")]
```

```
      kernel degree     gamma cost  cv_error         se
8 polynomial      2 0.1111111    1 0.2749182 0.00898764
```

```
one_se[order(one_se$cv_error), ]
```

```
      kernel degree     gamma cost  cv_error
8 polynomial      2 0.1111111  1.0 0.2749182
6 polynomial      3 0.1111111  0.1 0.2771388
7 polynomial      4 0.1111111  0.1 0.2771388
2     linear     NA        NA  0.1 0.2815334
```

Preferred model. The lowest CV error is the quadratic polynomial kernel with $C = 1$ (0.275, SE 0.009). It is not clearly better than its neighbors, though. Cubic and quartic kernels with $C = 0.1$ (0.277) and the linear kernel with

$C = 0.1$ (0.282) all fall within one standard error of it, and every candidate is only a few points better than always predicting "no CHD" (0.346). Raising C hurts the higher-degree polynomials (0.340 for degree 4 at $C = 10$), which points to overfitting. The radial kernel with $C = 0.1$ collapses to the majority class, and the best radial settings (0.286) do not beat the linear kernel. By the one-standard-error rule I prefer the **linear kernel with** $C = 0.1$. It is the simplest model whose error is statistically indistinguishable from the best. Its boundary is also a weighted sum of the standardized predictors, which is easy to interpret. The extra flexibility of nonlinear kernels does not pay off with 462 observations and nine predictors. If the goal were only the smallest CV error, the quadratic kernel with $C = 1$ would be the choice.

Question 2: SVM analysis skill

Invocation. I saved my skill as `.claude/skills/svm-classifier/SKILL.md` and applied it with a separate Claude Code agent working in an empty folder that held only `saheart.csv`. The data details were given in the prompt, not in the skill. This is the complete prompt:

> Invoke the skill at `.claude/skills/svm-classifier/SKILL.md`: read it and follow its instructions to analyze the data below. Let the skill guide every analysis and reporting choice.
>
> Data: `homework/week-09/q2-run/saheart.csv`. The response is `chd` (0/1). The predictors are the other nine columns: sbp, tobacco, ldl, adiposity, famhist, typea, obesity, alcohol, age. Code `famhist` as 1 for "Absent" and 2 for "Present".
>
> Use R (e1071 and ggplot2 are installed; Quarto 1.10 is available). Work only in `homework/week-09/q2-run/`. Produce report.qmd and render it to report.pdf, which must be exactly one page. Do not git commit anything.
>
> When you finish, reply with: the files you produced, the seed, the folds, the candidate settings, the selected model and why, the key numbers in the report, the page count of report.pdf, and anything in the skill you found ambiguous or had to decide yourself.

(Absolute paths in the prompt are shortened here to repository-relative paths.) The agent finished in about three minutes without steering. It used 10 stratified folds (seed 432), 25 settings (5 linear, 20 radial), and the one-standard-error rule, and it selected the linear SVM with $C = 0.1$.

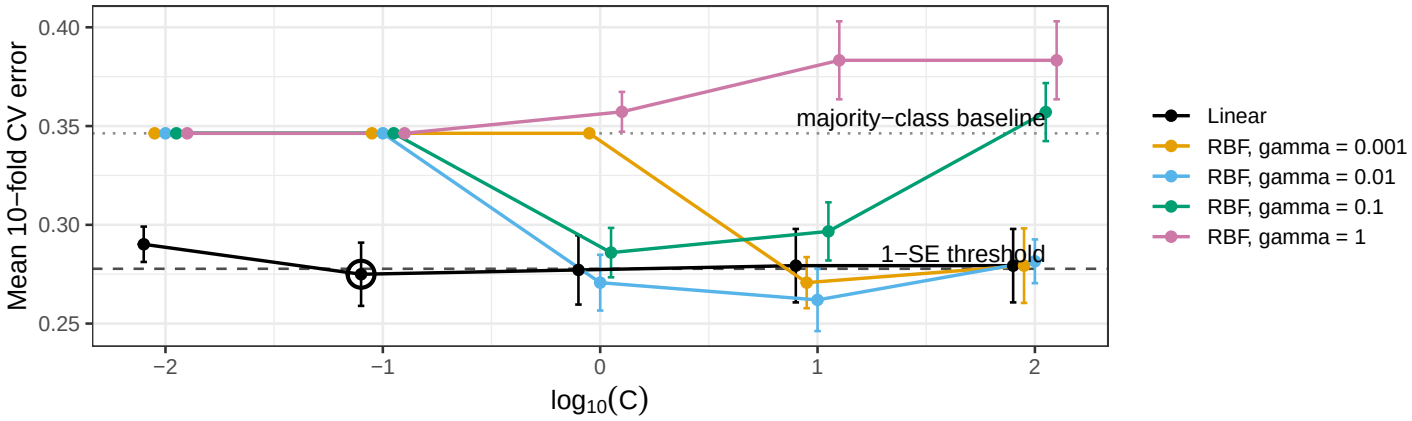
Check. I recomputed the selected and best settings with separate code on the same folds. The CV errors (0.275 and 0.262, both SE 0.016), the pooled out-of-fold sensitivity (0.500), specificity (0.844), PPV (0.630), and the nine linear weights all match the report. The agent noted choices my skill left open: it dropped $\gamma = 10$ to stay within 25 settings, ranked smaller C as simpler after "Linear < RBF", reported pooled out-of-fold metrics because there is no test set, and kept four table rows because the selected model ranked fourth. The selection agrees with Question 1, which also preferred the linear kernel with $C = 0.1$ under the one-standard-error rule.

The one-page, 11-point report follows on the next page, and the complete `SKILL.md` is appended after it.

SVM Classification of Coronary Heart Disease (SAheart)

Data and suitability. The SAheart data have $n = 462$ men and $p = 9$ predictors (sbp, tobacco, ldl, adiposity, famhist, typea, obesity, alcohol, age), so $n \gg p$ and training is fast. `famhist` is a two-level factor coded Absent = 1, Present = 2 (a single dummy, up to a shift). No values are missing. The response `chd` is 160 cases versus 302 controls (34.6% / 65.4%). That is milder than 80/20, so I left the hinge loss unweighted.

Method. I compared a linear kernel and an RBF kernel over a geometric grid of 25 settings: linear with $C \in \{10^{-2}, \dots, 10^{2}\}$ (5), and RBF with the same C values crossed with $\gamma \in \{10^{-3}, 10^{-2}, 10^{-1}, 10^{0}\}$ (20; $\gamma = 1/p \approx 0.11$ sits next to 10^{-1}). I used 10-fold stratified CV (seed 432, 46 or 47 observations per fold, 30–31 controls and 16 cases each). In every fold I standardized the predictors with the training fold's means and SDs only. The best setting was RBF ($C = 10$, $\gamma = 0.01$) with CV error 0.262 (SE 0.016), which puts the one-SE threshold at **0.278**. The simplest setting under that threshold, ordering Linear < RBF and then smaller C (stronger regularization), is the **linear SVM with** $C = 0.1$ (CV error 0.275 ± 0.016, rank 4 of 25).



Kernel	C	γ	Mean CV error (± SE)	Rank
RBF	10	0.01	0.262 ± 0.016	1
RBF	1	0.01	0.271 ± 0.014	2
RBF	10	0.001	0.271 ± 0.013	3
Linear	**0.1**	–	**0.275 ± 0.016**	**4**

Top-ranked settings by mean CV error (25 evaluated). Shaded row: selected by the one-SE rule.

Selected model, pooled out-of-fold predictions (positive class chd = 1)				
Error rate	Sensitivity	Specificity	PPV	Majority-class baseline error
0.275	0.500	0.844	0.630	0.346

Interpretation. The selected linear SVM misclassifies 27.5% of held-out men. Always predicting "no CHD" would misclassify 34.6%, so the SVM removes about 20.6% of the baseline error. It catches only 50.0% of true cases (sensitivity) while specificity is 84.4%. When it predicts CHD it is correct 63.0% of the time. The flexible RBF settings are not reliably better: the best one improves on the linear model by 0.013, less than one SE, and large γ overfits (error rises above the baseline at $\gamma = 1$). An SVM gives no probabilities or p-values. To see inside it, I refit the linear model on all standardized data and read its weights w (pull toward chd = 1 per SD): famhist (0.44), age (0.38), tobacco (0.34), typea (0.31), ldl (0.29), adiposity (0.14), sbp (0.13), alcohol (-0.04), obesity (-0.24). Family history, age, tobacco, type-A behaviour and LDL carry the largest positive weights, which matches known CHD risk factors. The negative weight on obesity partly reflects its correlation with adiposity, so it should not be read causally. The metrics come from the folds that also picked the model, so they are slightly optimistic.

Appendix: `SKILL.md`

```
---
name: svm-classifier
description: Guide end-to-end binary SVM analysis, tuning, evaluation, and single-page
 ↪ reporting.
disable-model-invocation: true
---
## 1. Suitability Assessment
Check that response is binary and sample size n aligns with predictor count p. Flag high n
 ↪ (>50k, slow training) or p >> n (overfitting risk). Encode categorical features via
 ↪ one-hot or dummy encoding. Confirm missing values are imputed or removed. Check class
 ↪ balance; note severe skew (>80/20) requiring class-weighted loss.

## 2. Kernel Selection
Evaluate two primary kernels: Linear and Radial Basis Function (RBF). Linear assumes
 ↪ approximately planar separation and serves as an interpretable baseline. RBF models
 ↪ smooth, non-linear boundaries. Limit candidate settings to a disciplined geometric grid
 ↪ across both kernels (<=25 total hyperparameter combinations).

## 3. Parameter Tuning & Preprocessing
Tune cost C across [10^-2, 10^2] for Linear; tune C and RBF bandwidth gamma across [10^-3,
 ↪ 10^1] (or default 1/p). Wrap all feature standardization inside CV folds (e.g., via
 ↪ pipelines) so test folds remain strictly unexposed. Fix all random seeds to guarantee
 ↪ deterministic fold splits.

## 4. Evaluation & Model Selection
Use 5- or 10-fold stratified cross-validation (matching class proportions across splits) with
 ↪ a fixed random seed. Report mean CV error and standard error (SE). To guard against seed
 ↪ instability, select the simplest model whose mean error falls within 1 SE of the best
 ↪ error (one-standard-error rule). Rank ties by complexity: Linear < RBF.

## 5. Visual & Tabular Artifacts
Figure: One compact line/facet plot showing mean CV error vs. log10(C), grouped by
 ↪ kernel/gamma, with error bars (+/-1 SE) and a dashed horizontal line at the 1-SE
 ↪ threshold. Table: Five columns listing Kernel, C, Gamma/Degree, Mean CV Error (+/- SE),
 ↪ and Model Rank, highlighting the chosen configuration.

## 6. Interpretation & Key Metrics
Report final test/CV metrics: overall error rate, sensitivity (recall), specificity, and
 ↪ positive predictive value. Benchmark performance against the no-information majority-class
 ↪ baseline. Acknowledge SVM black-box limits; interpret via linear weight magnitudes or
 ↪ predictor margin-distance sensitivities.

## 7. Single-Page Output Constraints
Format for standard letter page at 11pt font, 0.5-inch margins, single-spaced. Suppress all
 ↪ raw code blocks and console output. If space runs out, cut detailed parameter tables down
 ↪ to top-3 rows and shorten narrative interpretation, strictly preserving the core tuning
 ↪ plot, final summary table, and key metrics.
```